Journal of Electronic Testing
https://doi.org/10.1007/s10836-019-05779-y

RTOS Solution for NoC-Based COTS MPSoC Usage in Mixed-Criticality Systems

Serhiy Avramenko[1] · Massimo Violante[1]

Received: 30 June 2018 / Accepted: 22 January 2019

Abstract
Multi-core system on chip (MPSoC) is a clear trend for the future embedded systems. Network-on-chip (NoC) is the most scalable interconnection, allowing to have hundreds of cores on the same chip. On the other hand, safety critical applications (e.g., avionic) require the system to present a set of strict guarantees, which are difficult to achieve for MPSoC-based systems. In this work we target Commercial Off-The-Shelf (COTS) NoC-based MPSoC for the mixed criticality systems, in the safety critical field. The focus of the proposed work is on the issue of contention on the NoC and the related temporal interference. As the main contribution, we propose a partitioning technique to enable the usage of COTS NoC-based MPSoC for the mixed criticality systems, enabling an unbounded number of levels of criticality to be deployed. The proposed technique exploits the deterministic routing algorithm of the NoC and it is suitable for any NoC-based MPSoC which meets a set of fairly common characteristics. The partitioning technique is intended to have a purely software implementation as a module of a real-time operating system, which will allow easier certification as well. The proposed approach overcomes the concept of strict network partitioning between regions, as it implements traffic isolation allowing partitions to overlap. As a further contribution this paper describes the cost of proposed solution in terms of the network connectivity reduction. A set of rules to enable an efficient usage of the proposed solution has been provided.

Keywords Network-on-chip · Mixed-criticality systems · Commercial-off-the-shelf · Real-time operating system · Multi-core · Avionics

1 Introduction

Computer systems embedded in a controlled mechanical or electrical system (i.e., *embedded systems*) are nowadays used in almost every aspect of personal life, without even considering the industrial life [44]. Usually those systems need to process in real time the acquired system's parameters and to control the physical processes of the system.

Embedded systems are also used to deploy safety-critical applications, in fields like avionics, aerospace, automotive, medical, etc. Such systems are defined as systems which could have catastrophic consequences on human health, natural environment or infrastructure. Normally these systems are hard real time systems. This means that such systems have to be able to provide a correct behavior both in terms of computed outputs values and timing. Given the nature of a safety critical system, its developed is regulated by a set of standards aiming to certify its correctness.

The historical approach to ensure the correctness of a safety-critical embedded system was to avoid any non-functional dependencies among functions of a system. This has been done by creating a dedicated sub-system for each function, with no sharing resources among these sub-systems. In this way the absence of interference among functions was ensured by construction, which in turn was crucial to achieve the certification of such systems. An example of such approach is the *federated architecture* used in avionic field [49].

As the systems becomes more and more sophisticated with a growing number of applications, the one function – one computer approached was challenged, as integrating multiple function on a single sub-system provides clear advantages in terms of size, weight and power consumption (SWaP). Avionic industry is a clear example, as the weight and power

Responsible Editor: L. M. Bolzani Pöhls

✉ Serhiy Avramenko
serhiy.avramenko@polito.it

Massimo Violante
massimo.violante@polito.it

[1] Politecnico di Torino, Torino, Italy

Published online: 01 February 2019

dissipation are very important issues for this industry. The industrial research led to a number of solutions and standards allowing to integrate multiple applications, even having different safety requirements, in a shared computing platform [16]. The *mixed-criticality* systems, i.e., systems integrating applications with different requirements in terms of safety, do require a further effort to guarantee a bounded (or absent) interference between the applications, as the certification requirements go in tandem with development effort and thus the virtual absence of bugs assurance.

However, not only number of applications (for a given system) is ever-growing, so are the computational requirements for each of those applications. This makes the multicore architectures an appealing design choice for the next generation safety-critical embedded systems.

Multicore devices have been on the market for decades now and currently it is common to have tens of cores, alongside with a set of peripherals, integrated on a same *multi-processor system on chip* (MPSoC) device. The cutting edge *commercial off-the-shelf* (COTS) MPSoC devices normally make use *network-on-chip* (NoC) interconnection, as it is able to provide a virtually unbounded scalability when compared to the bus- and crossbar-based interconnections [19].

However, the usage of MPSoC in mixed criticality systems, especially NoC-based ones, is far from being a common practice. The reason of such reluctance is the complexity (and the related high cost) to ensure the absence of temporal interference between the applications. In fact, even to ensure a bounded temporal interference between the applications is still an unsolved issue. This implies that the correctness of the execution, from the time point of view, cannot be ensured either. The main source of such interference and thus, the main threat to the timing predictability, are the shared resources of the MPSoC. Two applications competing for a shared resource can delay each other in a very complex to predict way. In this work we will focus on the interconnection as this is a shared resource any MPSoC will for sure have.

Currently the certification standards for safety-critical systems refer to MPSoC as "highly complex" systems which discourages their exploration as a high complexity could easily lead to a very high certification effort [38]. Although there is a growing attention from the certification agencies side towards the usage of MPSoC systems for safety critical applications, e.g., for avionic domain.

This paper addresses the time predictability challenge related to COTS MPSoC usage in design of mixed-criticality systems. This work focus on the NoC interconnection as the source of time unpredictability. The hardware reliability is out of the scope of this paper, as the issue of ensuring bounded timing interference between the application is in place even considering a perfectly fault tolerant and reliable hardware. The proposed solution refers mostly to the avionic domain, while the considerations remain valid also for less stringent, in terms of certification, domains. We propose a technique, implemented as a module to be inserted in a certified *real time operating system* (RTOS), which implements isolation between applications running on a MPSoC. Moreover, we analyze the connectivity reduction cost of the proposed solution and derive some rules for application placement to mitigate such side effect. Finally, we experimentally demonstrate the validity of the proposed approach.

1.1 Mixed-Criticality System

The design of a safety-critical system, e.g., in avionic industry, has to follow a set of standards and recommendation. This is needed to make the aforementioned system be eligible to achieve the certification, indispensable for deploying the system. This certification process is mandatory as the failure of safety-critical system, by definition, potentially induce catastrophic effects on human health and the environment. Commonly, the certification standards classify applications into a number of different criticality levels, according to the severity of catastrophic consequences the applications can provoke in case of failure and also other aspects. The severity level expresses the effort required to achieve the certification of the related application. The applications having a high level of criticality undergo a complex and expensive certification process. For such applications, the development cost is usually smaller with respect to the testing and verification cost. On the other hand, applications having low level of criticality require a much cheaper certification process.

The most known standards are ISO 26262 [6] for automotive and DO-178C [7] for avionics, which define respectively four Automotive Safety Integrity Levels (ASIL) and five Design Assurance Levels (DAL). For instance, a DAL-A application might cause the aircraft to crash, while a DAL-D application, in the worst case, can just create some minor inconvenience.

Certification standards require that safety-critical applications, having different levels of criticality, do not interfere with each other. This means that the applications have to be isolated one from the other, both in space and time domains. If this isolation did not exist, it would happen that a low criticality application would be able to make its faults propagate towards a high critical application. This situation would not be acceptable as it violates the basic principle criticality levels: an application having a high level of criticality should be considered more bug free with respect to an application having a lower level of criticality. Considering each application). In

fact, without the proper isolation, for what it concerns the certification, all the applications should be considered as a unique "macro application" [39]. In fact, the presence of shared resources creates a channel for a fault to propagate from one application to the other. The simplest example is a faulty application, which generates a lot more traffic on the interconnection with respect to what was planned. This can easily result in a denial of service (DoS) for the other applications running on the MPSoC. This situation will be hardly feasible from the certification point of view, so conversely, the isolation will allow to certify each application according to its level of criticality.

To implement a proper isolation, normally the related mechanisms are implemented on both hardware and operating system level. For instance, in avionics, the *spatial* and *temporal* partitioning are defined by ARINC 653 [4]. This partitioning concept can be defined as "appropriate hardware and software mechanisms to restore strong fault containment" [45], where the restoring is referred to the federated architecture were the strong fault containment was assured by construction. Spatial partitioning enforces the isolation for what it concerns the data integrity, preventing an application from accessing the data of another application. This partitioning is usually implemented exploiting the memory management units (MMUs), which check whether the related application is allowed to access the requested memory address [30]. This feature is normally implemented by the RTOS used by the system [11]. Temporal partitioning enforces the isolations for what it concerns the execution time of the applications running on the system. As multiple applications are running on the same systems, an unexpected incise of accesses to a shared resource (e.g., CPU, interconnection, cache) by one of these applications could easily slow down the performance of the other applications, which can result in timing violations for the latter.

The literature provides a number of solutions to implement the isolation and the related partitioning techniques to develop a mixed-criticality system based on COTS single-core processors. These solutions are largely adopted by the industry by several years now [37]. However, the MPSoC are much more complex systems compared and ensuring a proper isolation, especially for what it concerns the temporal isolation, is still an open topic.

1.2 MPSoC Usage for Mixed-Criticality System Design

Referring to the Section 1.1, concerning the development the next generation of mixed-criticality systems, the usage of MPSoC for can be considered as a further step in improving the SWaP of the system. We have discussed how the development of mixed-criticality systems based on COTS single-core processors is possible by providing a proper isolation between the applications. Several solutions have been standardized, providing hardware and software partitioning mechanisms able to ensure that any application (confined within the related partition) is not able to corrupt the data, nor to slow down the execution of any other application (i.e., to ensure spatial and temporal partitioning) [11, 37, 42].

The main difficulty to extend these solutions to MPSoC is the presence of shared resources, potentially contended by multiple cores. If the NoC interconnect is addressed, which is the trend for the MPSoC, the complexity grows even more. As for NoC there could be contention on bus even without a concurrent access to any shared resource (apart the NoC itself). Integrating high criticality applications and low criticality applications on the same MPSoC is a challenging task if the non-functional dependency between the applications (which exists by construction) is not properly addressed. The main issue is that lower is the criticality level, lower will be the trustworthiness about the related application to be virtually bug free. This means that an eventual failure of the low criticality application can exploit the shared resources (e.g., interconnection), which provide an interference channel, and propagate to high critical application. A simple example of this situation can be the one of low criticality level application to monopolize the interconnection, starving the high criticality applications which are no longer able to access any resource of the MPSoC, even if those resources are not shared. A precise analysis of the effect of interferences in such a system would require knowing details about the microarchitecture of the MPSoCs which are often not shared by the manufacturers.

For these reasons the usage of COTS MPSoC for the mixed-criticality system design is still an open topic, with no established industrial solution yet [18].

1.3 Avionic Standards

Considering avionic embedded systems – i.e., embedded systems deployed on an aircraft – a historic approach was to avoid any kind of non-functional dependencies between applications. This has been done by providing to each application a dedicated computer. In detail, this design approach is called federated architecture [49] and it is characterized by deploying each function (or a set of tightly dependent functions) on a dedicated *on-board equipment* (OBE) instance. Each OBE is an embedded system isolated from all other OBEs but for it concerns a dedicated communication channels, with the final aim to almost eliminate the shared resources.

This federated architecture approach become no longer feasible by the ever-growing performance requirements of modern aircrafts. The reasons are mainly the huge number of subsystems would require an unsustainable maintenance effort as well as a tremendous budget requirement in terms of SWaP. The solution developed to replace the federated architecture approach is the *integrated modular avionic* (IMA) architecture design pattern [41]. In such architecture, different applications can have a single shared resource, provided that no interference is possible in any way. To help developers implement an IMA-based application, the standard ARINC-653 [4, 42] was proposed. This standard, among other things, describes the *application programming interface* (API) which all compliant RTOSs should provide. In conjunction with the API standard it was possible to deploy multiple functions (even having different levels of criticality) on the same OBE. In detail, the scheduling should be based on a windowed timeline scheduling, in which each application has its dedicated window during which it can use the hardware.

The civil avionic industry adopted a set of two standards, the RTCA DO-178C [7] and the RTCS DO-254 [2], the first targeting the software development and the second – hardware development process. The key concepts used in these standards can be considered as derived from the core concepts defined by IEC-61508 [3]. This standard defines safety integrity level (SIL). The SIL is the level of certification attributed to a given module of the system. The DALs, defined for the avionics, can be roughly equivalent to the SILs. The definition of each DAL, based on the hazard analysis performed on the airborne equipment, is reported in Table 1.

Assessment of the DAL to be assigned to a new equipment is performed according to the ARP4761 [1] standard, which describes a process to asses safety of airborne equipment. Avionic software is often analyzed according to the IEEE Std 1228–1994 [5], which describes the contents of a software safety plan. The plan, proposed by the developer, is analyzed by the certification authority to whom the developer submits the design as part of the certification process.

Table 1 DALs are defined based on the hazard analysis

Hazard classification	DAL	Max probability (flight hour)
Catastrophic	A	1 × 10–9
Hazardous	B	1 × 10–7
Major	C	1 × 10–5
Minor	D	–
No Effect	E	–

1.4 Extensions and Contributions

1.4.1 Extension of the Previous Work

The work presented in this paper is an extension of work done in [22]. With respect to the aforementioned, in this paper we introduce the following contributions:

- The usage of more than two nodes per critical application have been evaluated. The rules to create such system without provoking a performance degradation have been derived. This will allow to cover much wider number of real use cases;
- Deployment of more than two levels of criticality is described. As the safety critical systems usually define four or five levels of criticality, in this work we explain how to design such systems applying the partitioning solution proposed in [22]. A set of scenarios with three levels of criticality have been evaluated to illustrate the related placement rules;
- The proposed technique has been improved as the physical links are considered to be implemented as two monodirectional links [23].

1.4.2 Contributions

To summarize, in this paper, we extend the analysis of the technique proposed in [22]. The overall contributions of this work are:

- We propose a technique to allow the usage of COTS NoC-based MPSoC for the creation a mixed-criticality system. The proposed technique has been developed as a software module to be interred in a certified RTOS. Of course, the RTOS, once changed, has to undergo the certification process as it will not be certified any more. However, this is normal, as the system has to be certified as a whole in any case. Furthermore, the proposed approach would allow easier reuse of the solution by reducing the certification effort which would be needed just once to certify the RTOS. This solution is based on traffic flows isolation and can be applied to a set of architectures. The proposed solution, from the QoS point of view, features two levels of service: guaranteed service (GS) and best effort (BE). However, the proposed solution allows an unbudded number of applications and criticality levels to be implemented. Each application, which is set to have GS level of service, is guaranteed to have bounded latency and

throughput. As the proposed approach guarantees that no interference from the other applications does exist;
- We analyze the limits of the proposed solution especially for it concerns the connectivity issue and draw some rules to allow an efficient usage of the proposed methodology.

1.5 Outlines

This paper is organized as follows. Section 2 presents a survey of the literature on mixed-criticality applications on both bus-based and NoC-based MPSoCs; Section 3 describes the proposed spatial and temporal partitioning approach, and its implementation in a software module to be integrated in a RTOS; Section 4 presents experimental evaluation of the proposed module; Section 5 draws some conclusions and outlines future works.

2 Related Work

A the NoC architecture represent a clear trend for interconnection used by the COTS MPSoC, it has received a lot of attention from the research community. The main issue concerning the mixed-criticality system development is the timing predictability. A number of solutions were derived from those created for bus-based MPSoC, some solutions specifically developed for NoC have been proposed as well.

The first class of solutions, time multiplexing solutions, use static scheduling to avoid interference among tasks [14, 46]. This is implemented by partitioning applications in local phases – during which a core runs code using just local data – and memory access phases – during which the application commits produced data and reads data for the next computation phase. Other approaches may add stricter requirements to the local phase, like forbidding interrupt service routines (ISRs) execution [40]. However, all time multiplexing approaches introduce long idle times, lowering the overall utilization. The second class of solutions, bounded interference solutions, are focused on a more precise worst-case execution time (WCET) analysis in the case of a multi-core system. The interference sensitive WCET (isWCET) [35, 36], considers the number of cores that are accessing a shared resource at the same time to evaluate the access latency to be considered in the WCET estimation. The use of performance counter can also introduce fault tolerance based on on-line detection and recovery [21]. Another class of solutions focus on system scheduling. The problem was first formulated in [48], which also introduced a first solution. The solution was proposed for single-core systems and was later extended for multi-core systems [10, 26, 27]. More recently, the focus moved from bus-based multi-core systems to NoC-based many-cores. The scheduling problem, in this context, moved from tasks to flows as in [17, 28, 33, 50]. Partitioning schemes were also proposed for NoC-based systems, by extending network interfaces by adding a dedicated channel for high-criticality traffic, to implement a time-triggered scheme of communication [9]. This solution grants a bounded latency on high-criticality traffic by using a contention-free channel. The rest of the traffic is scheduled to happen without interfering with the high-criticality traffic. Different solutions are based on the concept of quality-of-service (QoS) [29, 34], or use hardware-enforced segregation between a safety-critical domain and a non-safety-critical domain [31, 47].

Recently some solutions have been developed, specifically targeting some of the cutting-edge COTS MPSoC. For instance, a solutions aiming the worst case estimation was derived specifically for Kalray MPPA® 256 [20] and for Tilera Tile Processor [8]. Kalray MPPA® was also considered to develop a system scheduling based solution [13], which creates a contention free scheduling. However, this kind of solutions relates on special hardware present in Kalray MPPA® 256. For the Tilera Tile Processor a partitioning based solution, like those described in [31, 47], was also presented [24]. There are also some companies, like Collins Aerospace, that claim they are near to achieve a certification of a COTS MPSoC-based system. In detail, they state that they expect an avionic certification of NXP T2080-based system to be achieved by the end of 2019 [43].

On the other hand, the certification authorities constantly increase their attention towards COTS MPSoC. Considering the avionic field, in 2017 Federal Aviation Administration (FAA) of the United States, issued a report to document the issues related to software assurance applied to multicore processors [32]. The report describes the interference-aware safety analysis and identifies three sequential steps: *the identification of an interference path; performance of an interference analysis to tag each interference channel as acceptable, unacceptable, unbounded, or faulty; and determination of interference mitigation*. However, to the best of our knowledge no COTS MPSoC-based systems certified for avionics do exist so far.

Recent studies also show how the soft errors can be an issue [15]. Several works which propose a solution do also exist. However, the issue of the time interference has to solved even considering the hardware to be perfectly resilient. In avionic field, the critical systems are developed in N-versioning. This means that the whole system is replicated to have redundant spare systems. So, in case of a node or a link failure, what would be normally done is the activation of a

spare unit to take the place of the faulty one. In the meantime, the faulty unit is reset, with the intention to recover from a soft error. If the faulty system will not recover, the aircraft will replace the faulty system.

Although there are many solutions proposed to solve the temporal interference issue, most rely on a special hardware or NoC architecture to be implemented. However, it is likely that, as it was for bus-based COTS MPSoCs, also NoC-based COTS MPSoCs will be optimized for the average, generic workload use-case which is far from the requirements of safety-critical applications. In bus-based MPSoC, a set of solutions to work around the non-determinism of their behavior had to be developed; predicting that the same kind of solutions will be in the end adopted for NoC-based COTS MPSoCs, this paper proposes a first step towards a complete solution. The work we propose in this work is somehow close to the one proposed in [24], as both are based on critical regions isolation. However, the solution we propose is much more flexible in terms of critical region constrains. Furthermore, our solution allows the detection of the faulty behavior and prevents the packet flooding phenomena.

3 Proposed Solution

The proposed solution solves the temporal and spatial partitioning, as described in Section 1.2, resorting to software means only – without any dedicated hardware support needed. The strong point of the proposed solution is to overcome the strict partitioning (e.g., the one described in [31, 47]). The proposed solution has been implemented as a software module to be integrated inside a certified RTOS.

In this work we will consider the following notation:

- N_k – node k;
- *app_cl_i* – application(s) having criticality level *i;*
- *app_cl_j* – application(s) having criticality level *j,* where *j* is different with respect to *i.*
- a node having a criticality level i – a node used by an app_cl_i;

This section will describe the proposed partitioning approach and the underlying NoC model characteristics required for the application of such approach. The software module we developed to implement the proposed approach is then described in detail. The considerations regarding the performance overhead of the proposed solution, as well as those related to the certification cost are done at the end to the Section 3.3.

3.1 NoC Model

The proposed approach is targeting simple NoC architectures, featuring a single physical network. This is done because the simplest is the architecture, the lower is the certification effort. In case one or more virtual networks are present, our solution is perfectly usable by simply not using the virtual networks.

The scheme of the considered NoC model is presented in Fig. 1. A node of the NoC is formed by a *processing element* (PE) connected to a *router* (R). The NoC is made of nodes connected by the two monodirectional links. Nodes are addressed through a dedicated memory map. Each node has its own memory map, including both local and remote resources. Local resources are accessed as usual through the local bus, whereas remote resources are forwarded on the NoC through the local *network interface* (NI). The NI is a peripheral on the local bus. It receives requests from the local PE according to the protocol defined on the local bus. The PE can then wait for a response (bus-transaction (BT) model) or it can continue its computations, pending an interrupt that signals that requested data is available at a predefined location on a local memory (direct memory access (DMA) model). The NI performs the following tasks:

1) Receives requests from the local PE to remote resources, transforms them in NoC packets, and injects packets inside the NoC;
2) Receives responses from remote resources and sends them to the local PE according to the BT model or to the DMA model;
3) Receives requests from remote nodes and forwards them on the local bus to the local PE. The local PE should

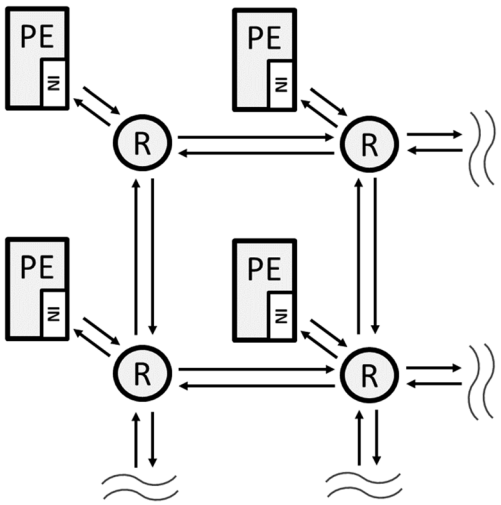

Fig. 1 NoC-based MPSoC scheme. A top left 2 × 2 sub-network is represented

respond to such requests according to the BT model or to the DMA model.

The NI has a look-up table to translate each global resource's address from the local memory map to a set of coordinates that identify the position of that resource on the NoC.

The PE in this NoC model is either a passive slave or a processor. The NI is in charge of making NoC transparent to the PE it is connected to.

It is worth to notice that, although the 2D mesh topology is the only considered in this work, the proposed solution is not theoretically limited to the former topology only.

3.2 Proposed Partitioning Technique

In order to ensure the absence of temporal interference between two applications having different levels of criticality, we propose a solution that can be classified as a patriating technique. The partitioning scheme we propose ensures the traffic isolation between applications having different level of criticality. In practice, we ensure that the traffic having a given level of criticality will never interfere with the traffic having a different level of criticality. This partitioning technique has been implemented purely in software in order to target COTS components. This work extends the technique presented for two levels of criticality [22], in order to be applied to multiple (virtually unbounded) levels of criticality. The actual implementation is described in the Section 3.3.

The proposed partitioning technique overcomes the classic strict domain segregation scheme (e.g., those presented in [31, 47]) as it allows a node to be simultaneously traversed traffic having different levels of criticality (e.g., by critical and non-critical traffic). This is a strong point of the proposed solution as it allows a much higher flexibility and thus a much better system utilization. This improvement can hardly be expressed by some metric as the proposed approach allow the critical regions (defined further on in this section) to overlap, which is not allowed for the strict partitioning techniques like those described in [31, 47]. Thus, the improvement, the proposed technique grants, is to make possible many placement scenarios which are not allowed for the strict partitioning. Thus, the proposed solution allows more applications to be deployed with respect to the strict partitioning -based techniques.

The proposed partitioning techniques can be seen as a set of components: traffic isolation enforcement (or traffic filtering), traffic redirection for the non-critical nodes and the system placement which implements the traffic partitioning. These components are not independent; however, they can be considered separately as their goal differs. First of all, the actual partitioning is done during the system placement phase. During this phase the routing algorithm, used by the NoC, is considered in order to derive a placement which ensures that the traffic having a given level of criticality is isolated from any traffic having a different level of criticality. The exception is done for the non-critical traffic, as the redirection module allows to change the routing for non-critical applications, so the placement takes into account this routing algorithm extension. This module has been developed to improve the system utilization. Finally, the traffic isolation enforcement technique monitors the NoC traffic paths and enforces the traffic paths to actually be those validated during the system design phase (which contains the placement phase). The rest of this section will describe the three components of the proposed solution. The actual implementation is described in Section 3.3. Section 4 provides an example of the proposed technique deployment, the cost of the proposed technique is discussed, thus justifying the solutions that are here anticipated.

The placement phase is the first to be performed. There are two goals to achieve during this phase. First goal is to attain a spatial isolation, which is done by not deploying any node which is shared by two applications having different levels of criticality. The second goal is to achieve the timing isolation, which is well known to be the main issue. This goal aims to achieve a system, in which the traffic traversing the NoC have no chance to interfere if that traffic belongs to applications having different levels of criticality. It worth to anticipate that for the proposed solution, the placement phase is not sufficient to achieve the time isolation. In fact, the traffic isolation enforcement (implemented as traffic filtering) is used to ensure that for any app_cl_i and app_cl_j, their traffic will actually never interfere. Where, the traffic of an application interferes with the traffic produced by another application if and only if both traffic uses the same physical link. The detailed discussion of the placement algorithms is out of the scope of this work. However, it should be noticed that the proposed approach is based on reserving physical links to a given level of criticality. As the proposed solution does not modify the routing algorithm for the critical applications, the higher is the number of reserved by app_cl_i, the more difficult will be for the app_cl_j to create a feasible placement. Thus, the placement algorithm should find a placement, which for any criticality level i will minimize the number of physical links which are reserved by app_cl_i. This, of course, is not true for the non-critical applications as they are supposed to use what remains after all the critical applications have been placed. Considering a *critical region* of app_cl_i as the union of the paths traversed by the traffic generated by app_cl_i, the placement algorithm should try to find a placement which minimizes the size of each critical region. It is worth to

mention that the rule of thumb to achieve the small size of critical regions is to minimize the distance between the nodes which should communicate with each other. As the placement algorithms already tries to minimize this distance (for both communication speed and power consumption), this is not a new feature out solution requires from the existent placement algorithms. However, when more than two nodes per critical application are considered, some additional considerations can be done, based on the routing algorithm used by the NoC, as described in Section 4.

The redirection feature is implemented to allow the non-critical nodes to better use the system resources. As we already discussed, the placement is done for the critical nodes first, thus the non-critical applications will use what remains. This means that many physical links will be probably non-available as they will be reserved by the critical applications. A non-critical application is generally not hard real time and in any case, we can afford a deadline violation for such application. Thus, we can afford to sacrifice some non-critical bandwidth by implementing the redirection, i.e., to make non-critical nodes able to re-transmit the traffic. This will de facto upgrade the routing algorithm and considerably raise the probability that, despite mane physical nodes are not available, the nodes are still able to communicate with each other. When placing the non-critical applications, the placement algorithm should take into account the redirection feature of the non-critical nodes.

Once the placement has been done, the traffic isolation enforcement (or traffic filtering) feature ensures that any not hardware -related problem (bug, DoS attack, etc.) will be able to break the traffic isolation by generating traffic paths violating the non-interference rule. This is done by filtering the faulty traffic and thus by preventing such traffic from being injected inside the NoC.

In order to explain the proposed solution, we propose the simplest example of a mixed criticality system: one critical application and one non-critical application. The considerations that will be done for this example are valid considering any two different levels of criticality, as far as the redirection feature is only applied to non-critical nodes. In case more than two levels of criticality are present then the considerations, explained by the example, are applied to each couple of different levels of criticality. The traffic with the highest level of criticality is not filtered because the design of such applications is supposed to (virtually) grant the absence of bugs [7]. In detail, the idea behind the proposed filtering approach is to exploit the knowledge of a deterministic routing algorithm used by the NoC; since deterministic routing algorithms assure that the path of a packet is completely specified by coordinates of source and target node pairs. Considering a NoC consisting of a single physical network (which is the simplest architecture possible), the following requirements must be satisfied:

1) the NoC should use a deterministic routing algorithm;
2) the routing algorithm should be known;
3) the topology of the NoC should be known.

It is worth noticing that despite we will only consider XY routing, our approach is valid with any deterministic routing algorithm, as far as requirements listed above are satisfied. Furthermore, despite this work will present examples of 4 × 4 NoC size, the proposed approach is applicable to any NoC size.

The requirement of a deterministic routing algorithm could be considered to be an important limitation, as dynamic reconfiguration solutions cannot be used. However, this is not an actual limitation as far as the avionic domain is considered. In fact, for the safety critical applications in general and especially for avionics, a dynamic reconfiguration is unfeasible as each possible configuration must be certified separately (and of course the sub-system in charge of reconfiguring the system has to be certified as well).

3.3 Proposed Software Module

The proposed filtering and redirection techniques are intended to be implemented as purely software module, to be integrated inside the a RTOS running on a NoC-based MPSoC. The proposed module (detailed description will be done further on) acts as a driver for the NoC interconnect, a) supervising and eventually filtering software requests for the NI; b) implementing the redirection feature for the non-critical nodes. Knowing both the routing algorithm and the topology of the network, the designer can configure the proposed software module to implement the proposed filtering and redirection approaches.

The filtering feature, of the proposed software module, is in charge of enforcing the traffic isolation on the NoC, as it was intended during the placement phase. The related sub-module oversees and manages communications that each active node of the NoC is attempting to initiate, with the exception done to the nodes belonging to the highest level of criticality (as the related application can be assumed to be virtually bug free [7]). In detail, as already explained in detail in Section 3.2, if an app_cl_i is attempting to inject a packet inside the NoC, which path would overlap the traffic path of an app_cl_j, this will violate the non-interference rule. Thus, the filtering sub-module will discard this packet, preventing this faulty packet from entering inside the NoC.

Eventually this faulty situation can be signaled to a monitoring application if any. The main goal of this feature is clear considering a simple system featuring one critical and one non-critical application. The goal is to filter all the traffic flows starting from a non-critical node which would interfere with the critical traffic. The former condition is detected by only considering the destination of a packet. As the proposed module runs locally and it does not need any synchronization with other modules on the NoC, therefore it does not add any communication overhead.

To better explain the proposed filtering sub-module we will consider an example of mixed-criticality system deployed on 2D 4 × 4 mesh topology using an XY routing. For the sake of simplicity, we will consider the simplest case of such system: one critical application and one non-critical application. Furthermore, the critical application will be using only two nodes, while the non-critical application will use all the remaining nodes. It could be misleading to extend this simple *critical* vs *non-critical* example to a system deploying multiple levels of criticality, thus the latter situation will be described further on in this section with a dedicated example. The case a critical application makes use of more than two nodes is considered further on as well.

Figure 2a depicts a placement scenario, we refer to as sc_1_1, of the mixed-criticality system we described in previous paragraph. This scenario is a worst case, as the two critical nodes, namely nodes N_0 and N_{15}, are at the maximum distance (considering the given routing algorithm used by the system) from each other. Although, this is an unlikely scenario, it can still happen. In fact, the NoC-based MPSoC are normally heterogeneous systems and in case more levels of criticality are deployed, it could happen that the only available instances of a given resource are far one from the other. The hotspot issue can also lead to this worst-case scenario. For more detail, the reader is invited to refer to the placement considerations done in Section 3.2.

As already stated in Section 3.2, the proposed partitioning solution overcomes the strict partitioning between critical and non-critical regions. Thus, the proposed partitioning technique does not forbid a non-critical node to be located inside a critical region as far as the traffic will not interfere with the critical traffic. As we will see further on in this section, the non-critical traffic benefits from the redirection feature provided by the proposed approach, thus it is much easier for a non-critical node to not interfere with the critical traffic.

Algorithm 1 describes the core behavior of the filtering sub-module. It implements the main functionality of traffic filtering according to a configured destinations map. The connectivity table is computed off-line according to the routing algorithm, the topology and the position of critical nodes. The connectivity table can be created by following a set of rules. In order to avoid the confusion, the rules will be listed also considering a system implementing multiple levels of criticality, while the simple example of Fig. 2a will

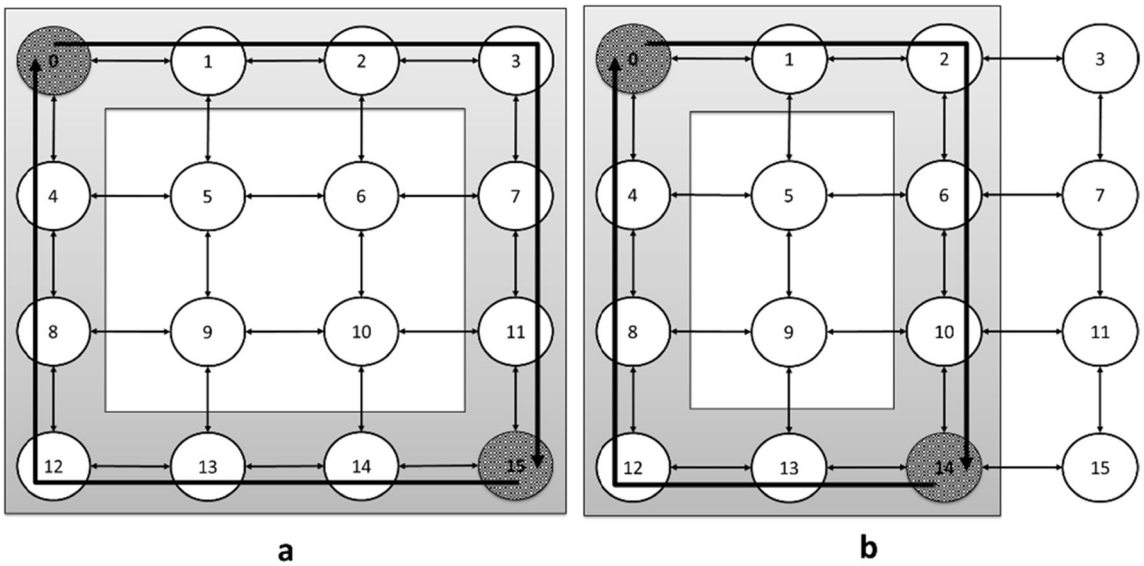

Fig. 2. 4 × 4 mesh topology with two critical nodes and the critical traffic paths (gray shade) defined by the XY routing path; a) sc_1_1: critcal nodes N0 and N15; b) sc_1_2: critical nodes N0 and N14.

require only a subset of these rules. The rules to be considered when defining the connectivity table are:

1) A node can only communicate with the nodes having the same level of criticality. This communication shall not violate rule 2);
2) A node can send a packet to another node (having the same level of criticality) if and only if the path of the packet will not overlap any path of the traffic belonging to any other criticality level. Where, two traffic paths are considered to be overlapping if and only if they share a physical link;
3) Only non-critical nodes can forward a (non-critical) packet. The forwarding shall not conflict with the rule 2). A packet can be forwarded an arbitrary high number of times.

Algorithm 1 Traffic filter (without redirection)

```
Input: dst_addr, payload;
Data: connectivity_table

if connectivity_table[dst_addr].ALLOWED then
   forward packet to NI
else
   drop packet and return error code
end if;
```

Applying these rules to the N_2 of the sc_1_1 (Fig. 2a) yields the connectivity table described in Table 2. It worth to be noticed that the assumption to have two monodirectional links (as connection between the routers) ensure that N_2 can communicate to N_{12} without any redirection is required. This is because the traffic will flow in the opposite direction with respect to the critical traffic, thus no physical link will be shared between the two. It can be observed that the N_2 (without the redirection feature) is not able to communicate to three other non-critical nodes. This is due to limitations imposed by its position within the critical region and the XY routing. This problem can be solved by adding a redirection functionality to the non-critical applications. This redirection functionality should act after the traffic filter and should use a second field in the connectivity table, as showed in reported connectivity tables. Such field is used as described in Algorithm 2 which extends Algorithm 1.

Algorithm 2 Traffic filter with redirection

```
Input: dst_addr, payload;
Data: connectivity_table

if connectivity_table[dst_addr].ALLOWED then
       if connectivity_table [dst_addr].RDR > -1 then
      generate redirection packet
      forward packet to NI
       else
              forward packet to NI
       end if;
else
   drop packet and return error code
end if;
```

Algorithm 3 Redirection packet check

```
Input: packet

if packet.redirection then
   extract the actual packet
   traffic_filter(actual_dst, actual_payload)
else
   forward to application software
end if
```

Table 2 sc_1_1: connectivity table (w/ and w/o redirection) for N_2

Destination node	With redirection		Without redirection
	Allowed	Redirection node	Allowed
0	0	X	0
1	1	−1	1
3	1	6	0
4	1	−1	1
5	1	−1	1
6	1	−1	1
7	1	6	0
8	1	−1	1
9	1	−1	1
10	1	−1	1
11	1	10	0
12	1	−1	1
13	1	−1	1
14	1	−1	1
15	0	X	0

The redirection packet is a special packet to be forwarded to the node indicated in the connectivity table. The information on the original destination is contained in the payload of the redirection packet. When a node receives a packet, the software running on the node should check if the received packet is a redirection packet and forward it to the actual destination as described in Algorithm 3. Redirection is an optional feature which is only useful to improve reachability in a NoC. Using redirection has no impact on critical traffic, but greatly increases performance of non-critical nodes, allowing integration of additional applications.

So far only the simplest example of mixed-criticality system, having only two levels of criticality, has been considered. Although, the algorithms and the rules for the connectivity table inferring are the same already explained in this Section, it worth to consider an example of a system deploying multiple levels of criticality. The key point of the overall solution we propose is to deploy the critical applications first. Considering the QoS point of view, the non-critical applications are granted with GS level and this is done reserving physical links to avoid any interference. This means that the non-critical applications can only use the nodes and physical links which are still available after the critical application deployment. The proposed solution implements the redirection for these non-critical applications in order to raise the probability the logic connectivity between the non-critical nodes will still exist, despite the adverse placing scenario de facto created by the critical node placement.

In Fig. 3a an example a system with two critical applications and one non-critical application is shown. It worth to notice that there is no difference, for what concerns the proposed solution, either there is a single non-critical application or an arbitrary high number on non-critical applications. In fact, a non-critical application does not create no kind of isolated region and more non-critical applications can interfere with each other. Considering the QoS point of view, the non-critical applications are granted with BE level.

Figure 3a shows a placement scenario related to a system deploying multiple levels of criticality, as there are two critical applications and one (or more) non-critical application(s). The relates critical applications make use of more than two critical nodes as well. It can be observed how the proposed approach does allow the two critical regions to overlap, as N_5 is located inside two critical regions at the same time. This placement

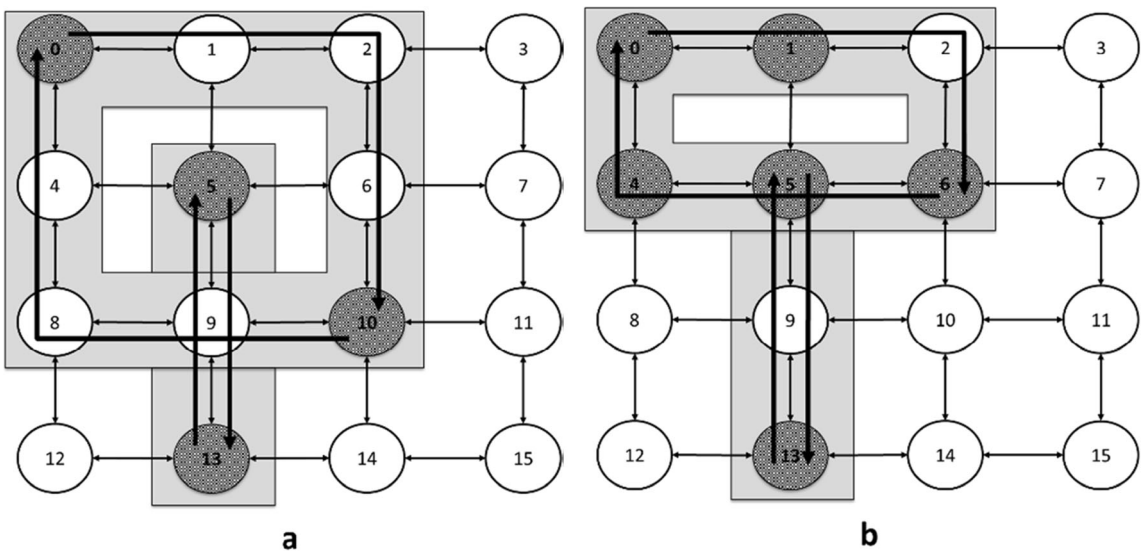

Fig. 3 Scenarios with two critical regions and more than two nodes per critical application; **a**) sc_2_1: critical region 1: nodes N0 and N10, critical region 2: nodes N5 and N13; **b**) sc_2_2: critical region 1: nodes N0, N1, N4 and N6, critical region 2: nodes N5 and N13

does not violate the requirement of overlapping traffic (different physical links are used), thus it is perfectly allowed.

Considering the case in which more than two nodes are used by a critical application, it is important to avoid a placing which will increase the size of the critical region. As we have seen in Section 3.2, this is a rule of thumb to avoid a high overhead due to connectivity reduction. In case the NoC has 2D mesh topology and XY (or YX) routing algorithm is used, we can observe that the critical region does not increase in size if:

- a node is added to a corner of the critical region;
- a node is arbitrary placed inside the critical region, as far as it only communicates to the nodes located in the same side of the rectangle (critical region);
- a node is arbitrary placed inside the critical region, if there are no nodes encircled by the critical region.

For instance, considering the application using N_0 and N_{10} of the Fig. 3a, more nodes can be added in the corners (N_2 and N_2) without affect the overall connectivity of the system. The nodes N_1, N_4, N_6 and N_9 can be also added without any connectivity reduction overhead, as far as they only communicate the nodes located in the same side of the rectangle. This means the, for instance N_1 should only communicate with N_0 (and N_2 if any). Figure 3b depicts another placement scenario which belong to the same example of mixed-criticality system as sc_2_1 do. In sc_2_2, both critical regions are in condition described by the last point of the list, as their critical regions does not encircle any node. This means that the critical nodes can be arbitrary located inside the critical region without no restrictions on its connectivity, for instance N_1 can communicate with all the nodes of its critical region. These considerations must be done during the placement phase, which aim to minimize the size of the critical regions.

From the performance perspective, the execution time of the proposed module should be predictable and as small as possible. This is true because, as we will see in this section, the filtering module we propose is extremely simple. The whole filtering process can be seen as consulting a bit of a table, according to the destination of the packet and discard a packet in case that bit is 0. The filtering module is executed for any packet to be transmitted through the NoC. This means that from the WCET point of view, we can just consider the transmission time to be increased by a small amount of clock cycles. In detail, considering the system without our solution implemented, a driver exists, which prepares the packet for the transmission and instructs the network interface to start the transmission. When our filtering module is present, that driver execution will be preceded by the execution of some lines of code checking the content of a table (which is our filtering module).

From the certification point of view, the RTOS will then require a new certification as modification will make it become non-certified. However, the proposed module can be seen as a driver, filtering the output traffic towards the (NoC) interconnection. This means that the effort to certify the RTOS obtained after our module is embedded, is much smaller than the certification of the RTOS from scratch. Furthermore, once certified it will be reusable in several applications without submitting it to a new certification process in each new application. To be certifiable, our module should be developed according to recommendations contained in [7]. The level of assurance should be selected according to the nature of the application, however, to be as general as possible, the module should be certified at the maximum level (DAL-A). Considering the effort for certification of a DAL-A module and the need for a low performance overhead, the implemented algorithm should be as simple as possible.

4 Experimental Evaluation

Experimental evaluation has been performed using a simulation environment considering a NoC model based on logic-based distributed routing (LBDR) [25] architecture. The considered NoC model is a simplified version of the NoC architecture presented in [12] and has 2D mesh topology, based on one physical network (no virtual network is present). This is a good model for the purpose at hand, as the proposed solution intentionally focus on low-complexity hardware, as our goal is to move towards a certifiable solution. Although, real-life NoC architectures used in COTS MPSoC, are more complex and may use different routing approaches, the proposed solution is applicable as far as it is possible to demonstrate that the extra complexity does not interfere with the proposed solution. The LDBR router is configured to implement an XY routing algorithm, which is a fairly common, deadlock-free routing algorithm.

The experiments have been done considering the sc_1_1 described in Fig. 2a. This scenario has been described in detail in Section 3.2. Destinations of non-critical traffic have been selected at random, also generating packets that interfere with the critical traffic. This is done in order to simulate the faulty behavior of an app_cl_i which interferes with the traffic of an app_cl_j, as this is exactly the issue the proposed solution has been developed to solve. Experiments were performed considering several packet injection rates (PIRs), defined as the number of packets injected in the NoC at each clock cycle by any given node. For each considered PIR, 10,000 packets

were injected by each node in the NoC. The experimental evaluation of the sc_1_1 has been done both considering the baseline setup and the setup where the proposed solution has been implemented.

The experimental results, related to baseline setup evaluation, are presented in Table 3. The figures exhibit the uncertainty in the communication latency among the two critical nodes (N_0 and N_{15}), as the non-critical traffic introduces congestion on the critical traffic path.

Once implemented, the proposed filtering sub-module enforces the reserved communication channel for the critical traffic (as shown in Fig. 2a). This traffic isolation has the effect to cancel any variance in the communication between the critical nodes. This is experimentally confirmed by evaluating the setup where the proposed solution has been implemented. The figures related to the latter setup are presented in the Table 4. As can be observed, the latency for the critical traffic to traverse the NoC is always 305 ns. The proposed approach cancels the latency variance, thus allowing an easy and provable WCET estimation for the critical applications.

The proposed solution, without the redirection feature, has the side effect of significantly reduce the logical connectivity for the non-critical nodes. The reasons of such reduction have been already explained in Section 3.2 and Section 3.3. In this work we define the *reachability* of a node N_i as the number of nodes to which N_i is able to send a packet. This metric must not be confused with the connectivity, which is usually defined as number of nodes to which a given node is physically connected. In fact, the existence of a physical connection between the two nodes is a necessary condition for these nodes to able to send a packet to each other, but it is not sufficient. A further requirement, to allow two nodes to communicate, is the routing algorithm to allow the logical connection for the given physical connection(s) existing between those nodes. In the reachability computation, critical nodes are not considered because the promise is that a placing for all the critical applications does exist, i.e., the placement algorithm is able to derive a placing for all the critical applications.

Reachability figures, without redirection module being implemented, are reported in Table 5. The average reachability obtained in this case is 10.7. This means that, on average, each node cannot send packets to 2 other nodes. The redirection extension is implemented with the purpose of improving the reachability for non-critical nodes, in order to have a more usable system while keeping warranties on the ability of the network of sustaining a critical traffic with deterministic latency. Implementation of the redirection feature, allows to obtain the perfect reachability, rising the average reachability to 13. The cost of this reachability improvement is some additional communication latency for non-critical nodes, due to the intervention of the software module during transmission.

A further placement scenario is presented in Fig. 2b. This configuration features two separated non-critical regions. However, the proposed approach allows the two regions to communicate with each other (even without redirection) as it does not imply strict critical and non-critical domain partitioning. Without the redirection module, the average reachability in this configuration would be of 11.9, which is higher with respect to the one of scenario of Fig. 2a, since the critical region is smaller. Using the redirection sub-module, the reachability increases up to the maximum value (13).

Considering the two placement scenarios of Fig. 3, with the redirection implemented, both will present the maximum connectivity for non-critical node. This means that despite the critical regions occupy the most of the NoC, each non-critical node is still able to communicate to each other critical node.

Although, the redirecting feature makes it harder to prevent the communication between non-critical nodes, this is still

Table 3 Critical traffic latency (ns): baseline setup

PIR	Max	Mean	STD.DEV.
0.02	815	334.8	55.6
0.01	665	315.1	30.4
0.007	655	311.6	24.4

Table 4 Critical traffic latency (ns): proposed solution implemented

PIR	Max	Mean	STD.DEV.
0.02	305	305	0
0.01	305	305	0
0.007	305	305	0

Table 5 sc_1_1: reachability for non-critical nodes (N_0 and N_{15} not considered): no redirection

Node ID	Unreachable nodes	Reachability
1	2, 3, 6, 7, 10, 11, 14	6
2	3, 7, 11	10
3	7, 11	11
4	11	12
5	11	12
6	11	12
7	11	12
8	4	12
9	4	12
10	4	12
11	4	12
12	4, 8	11
13	4, 8, 12	10
14	1, 4, 5, 8, 9, 12, 13	6
Average	N/A	6

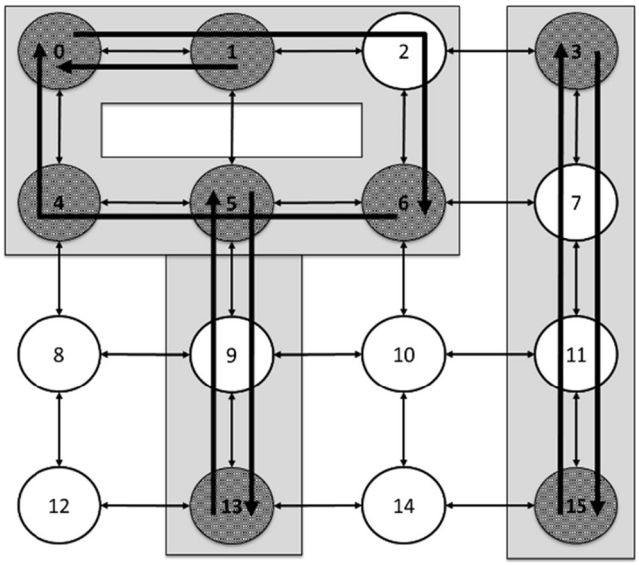

Fig. 4. sc_3_1: scenario with three critical regions and more than two nodes per critical application; critical region 1: nodes N0 and N10, critical region 2: nodes N5 and N13; critical region 3: nodes N3, N15;

possible. Figure 4 presents a system with a further critical application with respect to the sc_2_2, furthermore, the communication between N_1 and N_0 is explicitly shown. For this placing scenario, and because there is communication between N_1 and N_0, the effect is that N_2 is isolated from the other non-critical nodes. It worth to be noticed that in case there were no communication between N_1 and N_0, there would be no N_2 isolation. This shows how the rule to minimize the critical region is a rule of thumb.

The considered examples of placement, especially the last one, show the limit of the proposed solution. When the number of critical applications grow, some non-critical nodes can be prevented from communicate with each other; some non-critical node can be even completely isolated from the other nodes. In order to avoid this connectivity reduction overhead, there is a rule of thumb to minimize the size of the critical regions. However, as the sc_3_1 shows, this is just a rule of thumb. In the real application, we have a set of applications to be deployed, each with some resource requirements. On the other hand, the MPSoC has a set of resources and the premise is that these resources are sufficient to host the applications without the proposed approach. The applications have to be mapped on the available resources during the placement phase. At this phase, the placement algorithm should first of all find a placement for all the applications as described in Section 3.2 and Section 3.3. During this placement inference, the placement algorithm will try to find a placement which would avoid reachability reduction and non-critical node isolation. The detailed discussion on the placement algorithms is out of scope of this work.

5 Conclusion

This work describes a purely software solution to obtain a NoC configuration for a mixed-criticality system design. The proposed solution solves the issue of the spatial and timing interference of the shared NoC interconnection. In the scope of this work we considered the case of an avionic application, with the relevant standard being the RTCA DO-178C [7] and recommendations contained in [18] and similar documents.

The proposed partitioning technique, based on deterministic routing, has been implemented as software module. The proposed module is intended to be part of an RTOS in order to optimize the certification effort. Being a module of an RTOS means that the module should be certified only once together with the containing RTOS. The proposed module implements two functionalities:

1) traffic filter, to enforce an isolated communication channel for every level of criticality;
2) redirection, to allow a non-critical node to reach more nodes, solving the connectivity limitations introduced by the traffic filter functionality. This can be considered as an optional functionality, providing a higher usability of the NoC.

The proposed solution has been developed by software means only, in order to be applicable to the COTS components. Experimental results show that the latency determinism is granted thanks to the traffic filter module. The redirection module, only used for non-critical applications, grants the latter a better usability of the NoC-based system.

The proposed software module is implemented as few lines of code, thus expected to have a negligible performance overhead. However, in this work only the timing metrics of the NoC have been collected. The figures about the performance overhead will be part of our future work.

The technical limitations of the proposed approach are the routing algorithm used by the NoC to be both deterministic and known as well as the topology of the NoC to known. It worth noticing that the proposed solution is size independent, nor limited to mesh topology. The practical limitation of the proposed partitioning technique is the high number of critical nodes to be arbitrarily mapped. In fact, the proposed technique is based on the link reservation, thus the placing algorithm responsibility is to find a placing which minimized the size of each critical region.

References

1. (1996) ARP4761 - Guidelines and methods for conducting the safety assessment process on civil airborne systems and equipment. SAE International
2. (2000) RTCA/DO-254 Design Assurance Guidance for Airborne Electronic Hardware. RTCA Incorporated
3. (2000) IEC-61508. Functional safety of electrical/electronic/programmable electronic safety related systems. International Electrotechnical Commission
4. (2003) ARINC 653–1 avionics application software standard interface. Technical report. ARINC
5. (2010) IEEE Std 1228–1994 – IEEE Standard for Software Safety Plans. Institute of Electrical and Electronics Engineers
6. (2011) ISO 26262, Road Vehicles - Functional Safety. International Organization for Standardization (ISO)
7. (2012) DO-178C/ED-12C software considerations in airborne systems and equipment certification. RTCA, Incorporated
8. (2015) Tile Processor Architecture Overview for the TILEPro Series, Tilera Corporation, http://www.mellanox.com/repository/solutions/tile-scm/docs/UG120-Architecture-Overview-TILEPro.pdf. Accessed 12 June 2017
9. Ahmadian H, Obermaisser R (2015) Time-triggered extension layer for on-chip network interfaces in mixed-criticality systems. In Proc. 18th Euromicro Conf. Digit. Syst. Des (DSD), pp. 693–699
10. Anderson J, Baruah B, Brandenburg B (2009) Multicore operating-system support for mixed criticality. In Proc. Workshop on Mixed Criticality: Roadmap to Evolving UAV Certification
11. Avramenko S, Esposito S, Violante M, et al (2015) An hybrid architecture for consolidating mixed criticality applications on multicore systems. In Proc. IEEE 21st Intl On-Line Testing Symp (IOLTS), pp. 26–29
12. Azad SP, Niazmand B, Janson K, George N, Oyeniran AS (2017) From Online Fault Detection to Fault Management in Network-on-Chips: A Ground-Up Approach. In Proc. Design and Diagnostics of Electronic Circuits & Systems (DDECS), pp. 48–53
13. Becker M, et al (2016) Contention-free execution of automotive applications on a clustered many-core platform. In Proc. 26th Euromicro Conference on Real-Time Systems (ECRTS), pp. 14–24
14. Boniol F, Cassé H, Noulard E, Pagetti C (2012) Deterministic execution model on COTS hardware. Lect Notes Comput Sci (including Subser Lect Notes Artif Intell Lect Notes Bioinformatics), vol 7179 LNCS, pp. 98–110
15. Bortolon FT, Abich G, Bampi S, Reis R, Moraes F, Ost L (2018) Exploring the Impact of Soft Errors on NoC-based Multiprocessor Systems. In Proc. IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–5
16. Burns A, Davis RI (2017) A survey of research into mixed criticality systems. CSUR 50(6):82
17. Burns A et al (2014) A wormhole NoC protocol for mixed criticality systems. In Proc. IEEE Real-Time Systems Symposium, pp. 184–195
18. Certification Authorities Software Team (2014) CAST-32 position paper. Federal Aviation Administration / European Aviation Safety Agency
19. Dally WJ, Towles B (2001) Route packets, not wires: on-chip interconnection networks. in Proc. 38th Design Automation Conference, pp. 684–689
20. de Dinechin B D, van Amstel D, Poulhiès M, Lager G (2014) Time-critical computing on a single-chip massively parallel processor. In Proc Conference on Design, Automation & Test in Europe (DATE), pp 97:1–97:6
21. Esposito S, Violante M et al (2017) A novel method for online detection of faults affecting execution-time in multicore-based systems. ACM Trans Embed Comput Syst 16(4):1–19
22. Esposito S, Avramenko S, Violante M (2018) RTOS for mixed criticality applications deployed on NoC-based COTS MPSoC. In Proc. IEEE 19th Latin-American Test Symposium (LATS), pp. 1–6
23. Esposito S, Avramenko S, Violante M (2018) Efficient software-based partitioning for commercial-off-the-shelf NoC-based MPSoCs for mixed-criticality systems. In proc. IEEE 24th International Symposium on On-Line Testing And Robust System Design (IOLTS), pp. 189–194
24. Fattah M et al (2014) Mixed-criticality run-time task mapping for noc-based many-core systems. In Proc. 22nd Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP), pp. 458–465
25. Flich J, Duato J (2008) Logic-based distributed routing for NoCs. In proc. IEEE Comput Archit Lett 7(1):13–16
26. Giannopoulou G et al (2013) Scheduling of mixed-criticality applications on resource-sharing multicore systems. In Proc. Int. Conf. Embed. Software, pp. 1–15
27. Giannopoulou G et al (2014) Mapping mixed-criticality applications on multi-core architectures. In Proc. Des. Autom. Test Eur. Conf. Exhib. (DATE), pp. 1–6
28. Giannopoulou G et al (2016) Mixed-criticality scheduling on cluster-based manycores with shared communication and storage resources. Real-Time Syst 52(4):399–449
29. Goossens K, Dielissen J, Radulescu A (2007) Æthereal network on Chip: concepts, architectures, and implementations. IEEE Des. Test Comput. 22(5):414–421
30. Hattendorf A, Raabe A, Knoll A (2012) Shared memory protection for spatial separation in multicore architectures. In Proc. 7th IEEE International Symposium on Industrial Embedded Systems (SIES), pp. 299–302
31. Hollstein T, Azad SP, Kogge T, Niazmand B (2015) Mixed-criticality NoC partitioning based on the NoCDepend dependability technique. In Proc. 10th International Symposium on Reconfigurable and Communication-centric Systems-on-Chip (ReCoSoC), pp. 1–8
32. Hughes WJ (2017) Assurance of Multicore Processors in Airborne Systems. DOT/FAA/TC-16/51. U.S. Department of Transportation Federal Aviation Administration. https://www.faa.gov/aircraft/air_cert/design_approvals/air_software/media/TC-16-51.pdf. Accessed 1 December 2018
33. Indrusiak LS, Harbin J, Burns A (2015) Average and worst-case latency improvements in mixed-criticality wormhole networks-on-Chip. In Proc. Euromicro Conference on Real-Time Systems, pp. 47–56
34. Marescaux T, Corporaal H (2007) Introducing the SuperGT network-on-chip. In Proc. Design Automation Conference, pp. 116–121
35. Nowotsch J et al (2014) Multi-core interference-sensitive WCET analysis leveraging runtime resource capacity enforcement. In Proc. Euromicro Conf. Real-Time Syst., pp. 109–118
36. Nowotsch J et al (2014) Monitoring and WCET analysis in COTS multi-core-SoC-based mixed-criticality systems. In Proc. Des. Autom. Test Eur. Conf. Exhib. (DATE), pp. 1–5
37. Paulitsch M, Driscoll K (2014) Industrial communication technology handbook. Capter 48 SAFEbus. CRC press
38. Paulitsch M, Duarte O M, Karray H, Mueller K, Muench D, Nowotsch J (2015) Mixed-Criticality Embedded Systems – A Balance Ensuring Partitioning and Performance. In Proc. Euromicro Conference on Digital System Design, pp. 453–461

39. Paulitsch M et al (2015) Mixed-criticality embedded systems – a balance ensuring partitioning and performance. In Proc. Euromicro Conference on Digital System Design (DSD), pp. 453–461
40. Pellizzoni R et al (2011) A predictable execution model for COTS-based embedded systems. In Proc. Real-Time Technol. Appl., pp. 269–279
41. Prisaznuk PJ (1992) Integrated modular avionics. In Proc. Aerosp. Electron. Conf., vol. 1, pp. 39–45
42. Prisaznuk PJ (2008) ARINC 653 role in Integrated Modular avionics (IMA). In Proc. IEEE Dig. Avionics Sys. Conf
43. Radack D, Tiedeman H G, Parkinson P (2018) Civil certification of multi-core processing Systems in Commercial Avionics. Rockwell Collins. https://www.rockwellcollins.com/-/media/files/unsecure/products/product-brochures/computing/processing-mission-computers/multi-core-certification-white-paper.pdf. Accessed 1 Dec 2018
44. Rajkumar RR, Lee I, Sha L, Stankovic J (2010) Cyber-physical systems: the next computing revolution. in Proc. 47th Design Automation Conference (DAC), pp. 731–736
45. Rushby J (1999) Partitioning in Avionics Architectures: Requirements, Mechanisms, and Assurance. NASA Langley Research Center, NASA CR-1999-209347
46. Schranzhofer A, Chen J-J, Thiele L (2009) Timing predictability on multi-processor systems with shared resources. In Proc. Embed. Syst. Week-Workshop Reconciling Perform. with Predict., p. 89
47. Triviño F, Sánchez J, Alfaro F, Flich J (2012) Network-on-Chip virtualization in Chip- multiprocessor systems. JSA 58(3–4):126–139
48. Vestal S (2007) Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance. In Proc. Real-Time Syst. Symp, pp. 239–243
49. Watkins CB, Walter R (2007) Transitioning from federated avionics architectures to integrated modular Avionics. in Proc. IEEE 26th Digital Avionics Systems Conference, pp. 2.A.1-1-2.A.1-10
50. Xiong Q, Wu F et al (2017) Extending real-time analysis for wormhole NoCs. IEEE Trans Comput 66(9):1532–1546

Serhiy Avramenko , obtained his B.Eng. in 2012 from Università degli Studi di Siena and his M.Eng. in 2015 from Politecnico di Torino. Since 2015 he is a Ph.D. student in Politecnico di Torino, working on high-performance computing systems for harsh environments based on commercial-off-the-shelf components.

Massimo Violante , received the MS and Ph.D. from Politecnico di Torino, where he is now Associate Professor. Prof. Violante main research topics are design and validation of embedded system for safety- and mission-critical applications, with particular emphasis on the use of commercial off-the-shelf components like multicore processors and field programmable gate arrays in automotive and avionic applications. Prof. Violante published more than 150 papers in the area of testing and designing reliable embedded systems, and he co-authored two books. He served as program co-chair and genera co-chair of the IEEE Defect and Fault Tolerance in VLSI and Nanotechnology Systems for 2011 and 2012 and as Program Chair for the IEEE European Test Symposium for 2012. Prof. Violante is scientific responsible of a number of research projects in the area of embedded systems for space, avionic, and automotive applications, in collaboration with European Space Agency, Thalse Alenia Space, and Magneti Marelli.